



INFORMÁTICA II

Bioingeniería



Índice



Repaso



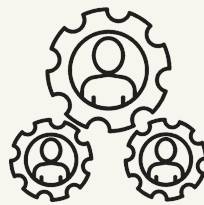
Diagrama UML



Desarrollo
Ejercicio



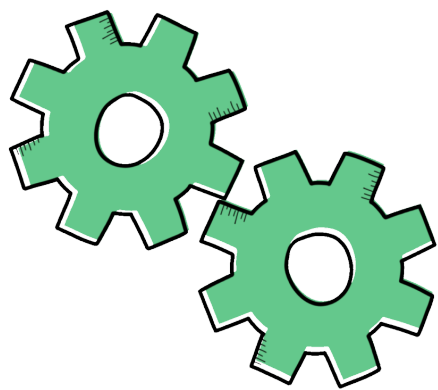
Dudas



Para la clase



**Problemas
adicionales**





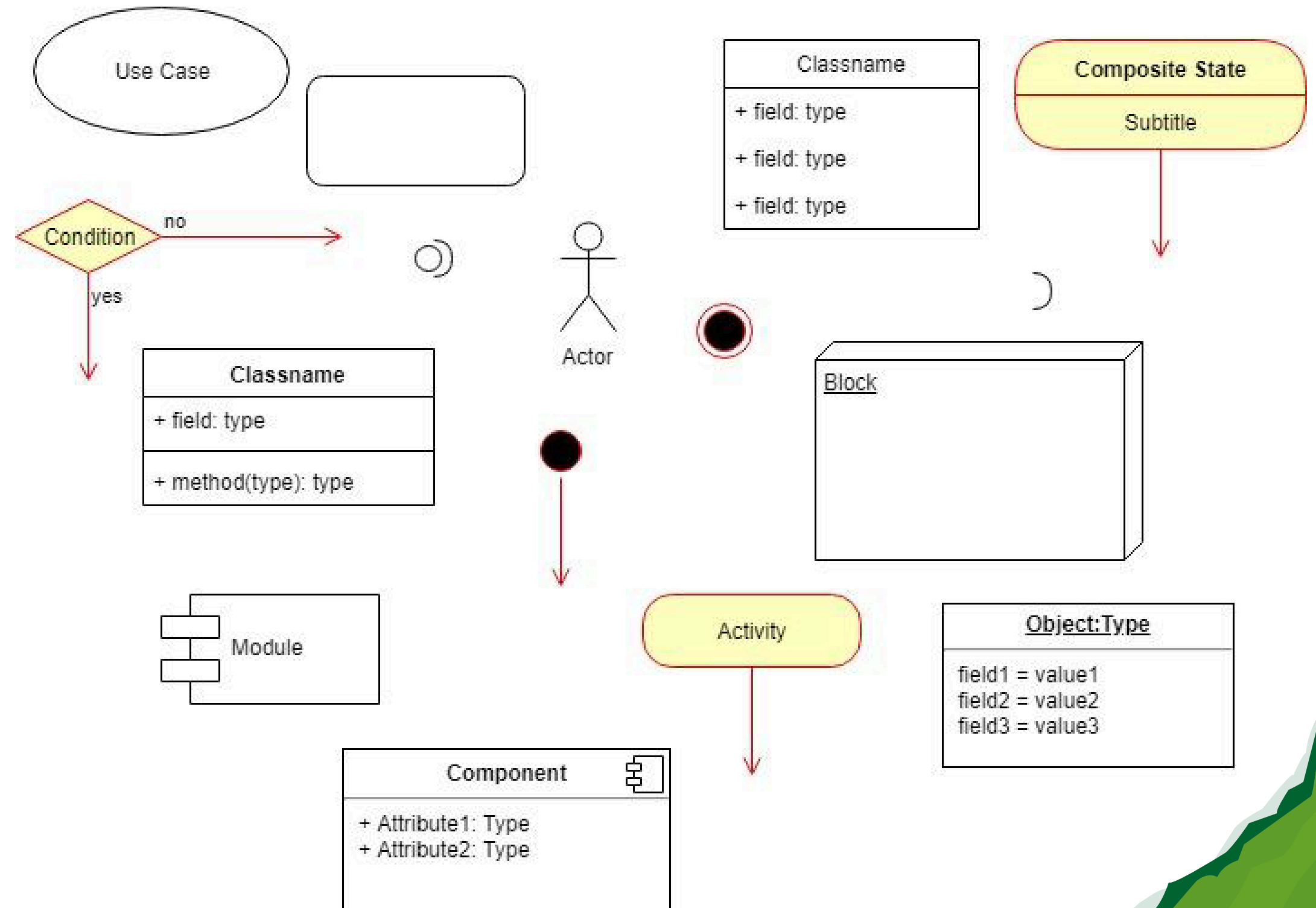
Repaso de conceptos de la clase pasada:

- En qué consiste la POO?
- Qué significa abstraer, cómo lo hace el paradigma **P** **O** **O** ?
- Cómo se trabaja la herencia en Python?
- En que caso se recomienda usar herencia?
- Qué es polimorfismo y como se usa en Python?



Notación UML

Fue diseñado para formar un lenguaje de modelado visual común (estándar), semántica y sintácticamente rico para arquitectura, el diseño y la implementación de sistemas de software complejos, tanto en estructura como en comportamiento





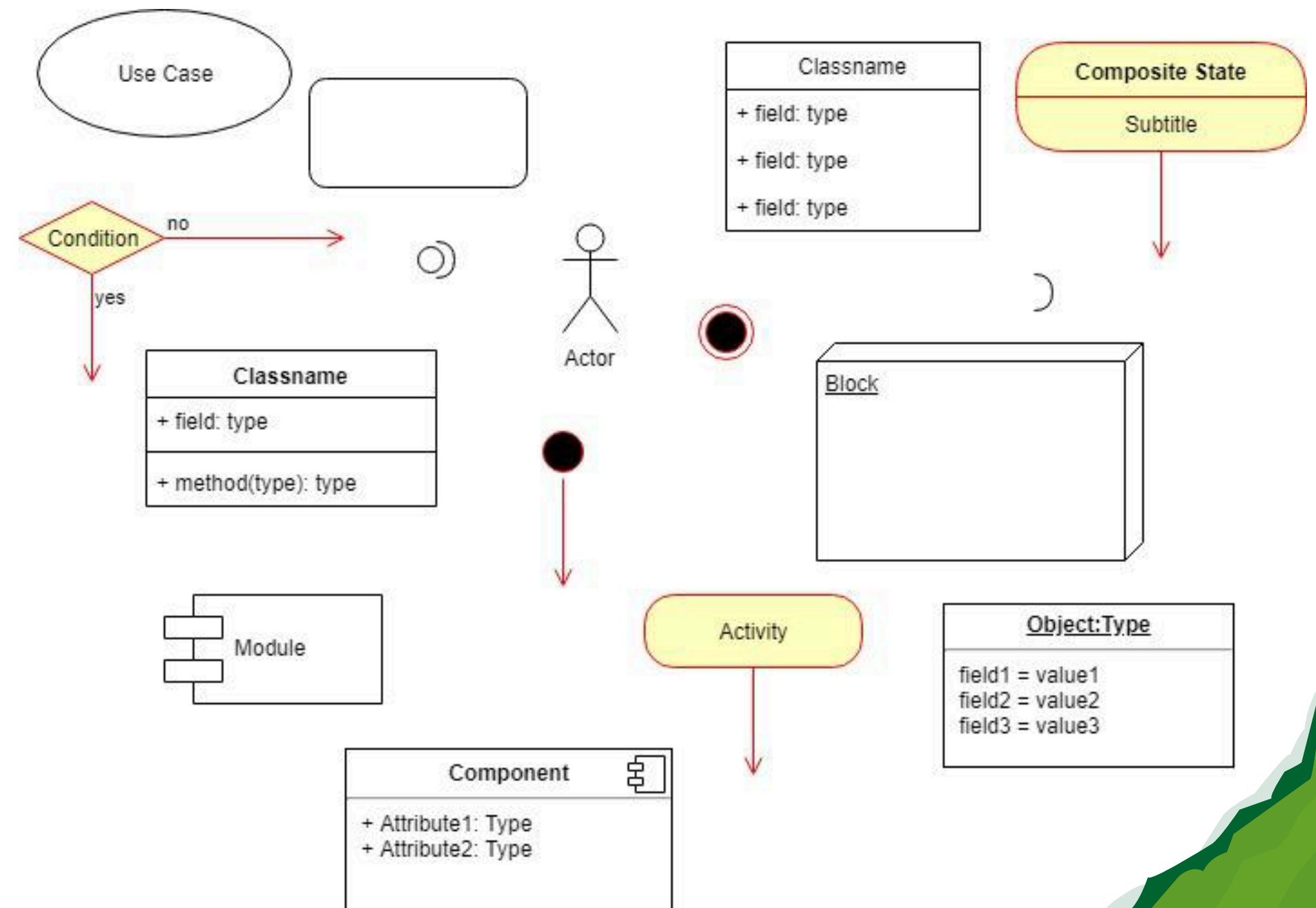
Diagramas de clase

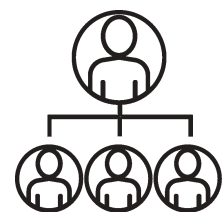
Clase: Es la estructura de un objeto, es decir, la definición de todos los elementos de que está hecho un objeto.

Objeto: Es, por tanto, el “resultado” de una clase.

Cada objeto (ó instancia) pertenece a una clase.

Ejemplos en python: Números, caracteres, tuplas, listas, diccionarios, etc.





Diagramas de clase

Cada clase es representada en un rectángulo con 3 compartimentos

Nombre de Clase
atributo: Tipo / atributo Derivado
operación()



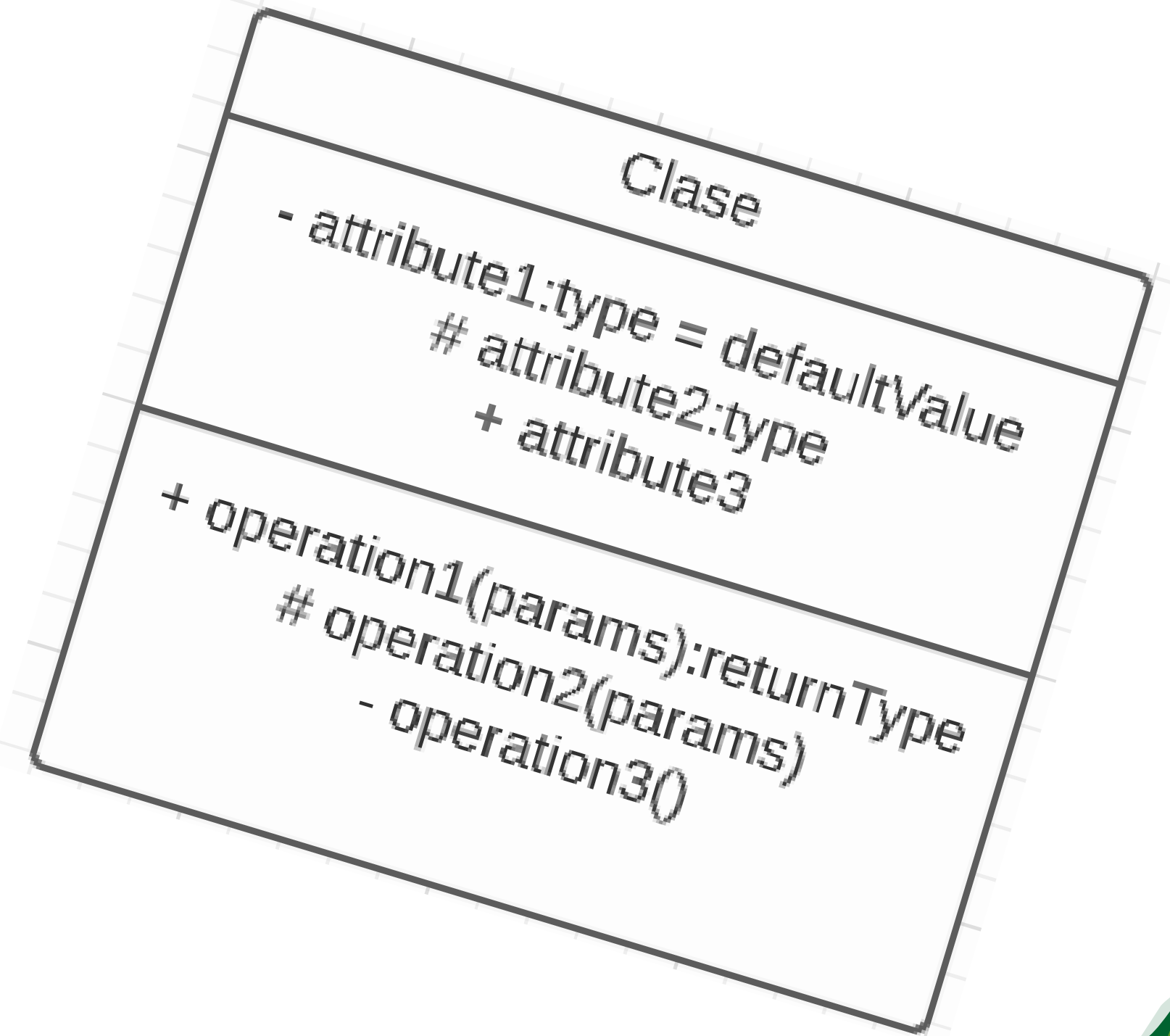
Aviones
modelo de avión cantidad de motores velocidad de crucero carga útil
acelerar () elevarse () girar () descender () desacelerar ()





Diagramas de clase

Los atributos se ubican en el segundo compartimentos y están precedidos por un signo (-,+,#) que indica si el atributo es privado, público o protegido respectivamente (ENCAPSULACIÓN), también puede tener información adicional de tipo , multiplicidad, valor.



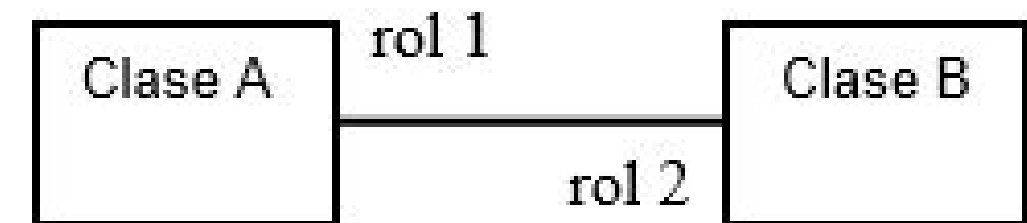
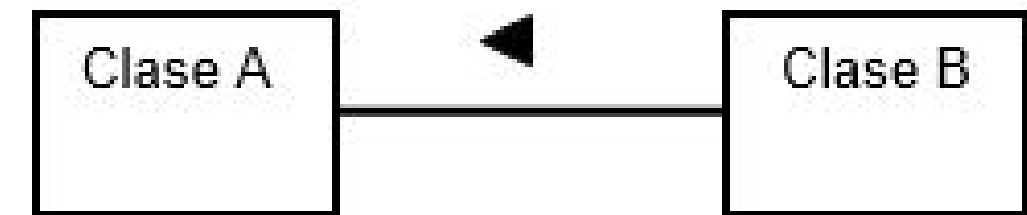
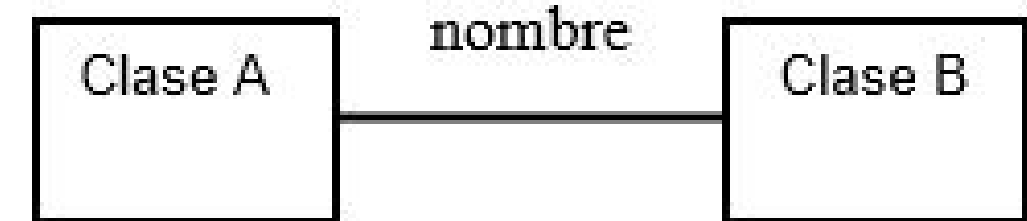
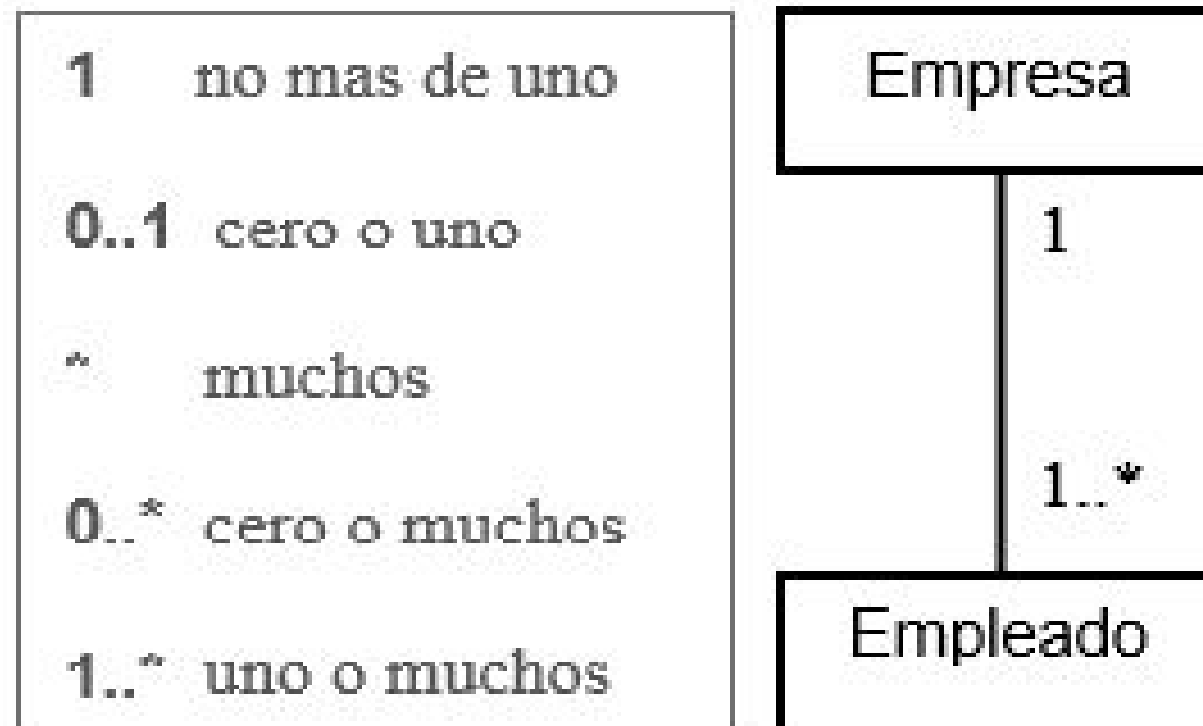


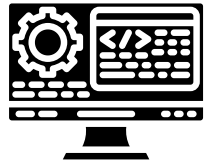
Diagramas de clase



Asociaciones: Tipo de relaciones (asociación, dependencia, agregación, composición) que pueden existir entre clases, desempeñando un papel(rol), una direccionalidad a otra clase.

Multiplicidad: Cantidad de ocurrencias que existen de un objeto en otro objeto





Primer ejercicio de abstracción

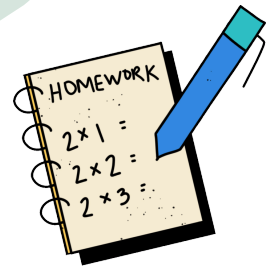


Se solicita construir un sistema que permita almacenar datos de pacientes, médicos y enfermeras.

Los pacientes tienen información de nombre, cédula, género y servicio, que corresponde al servicio donde están alojados.

Las enfermeras tienen la misma información que los pacientes, menos servicio, mas turno (que puede ser 7-19, 19-7, corrido) y rango (auxiliar, jefe, jefe del servicio).

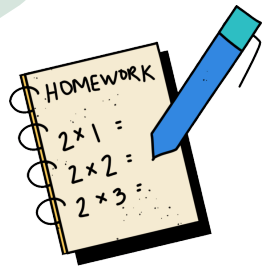
Adicionalmente pueden existir médicos que tienen toda la información de las enfermeras, menos el rango, más especialidad



Primer ejercicio de abstracción

Tips:

- Las clases suelen ser **sustantivos** que **agregan** un conjunto de propiedades y funcionalidades
- Los atributos suelen ser **sustantivos o adjetivos** que se pueden trabajar con variables más simples y hacen parte a su vez de clases.
- Los métodos suelen **ser verbos que indican funcionalidades** de una clase
- **Buscar relaciones de generalización para usar la herencia.** Recordar que la herencia permite partir de lo general a lo específico
- **No todos los atributos o métodos están definidos en el problema**, a veces es necesario definir



Primer ejercicio de abstracción

- **Clases (En singular e iniciando en mayúscula):** Paciente, Enfermera, Médico
- **Atributos:** nombre, cédula, género y servicio, turno, rango, especialidad
- **Métodos:** No hay ninguno definido en el enunciado
- **Relaciones de generalización:** Todas las clases en el problema en su forma más general son personas, así que podría haber una clase Persona con los atributos que son comunes a todas las clases de manera que las otras clases la hereden y añadan, o extiendan, nuevas funciones. A su vez, todos los empleados del hospital también podrían estar en una clase: Empleado_Hospital o EmpleadoHospital.

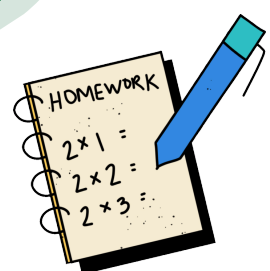
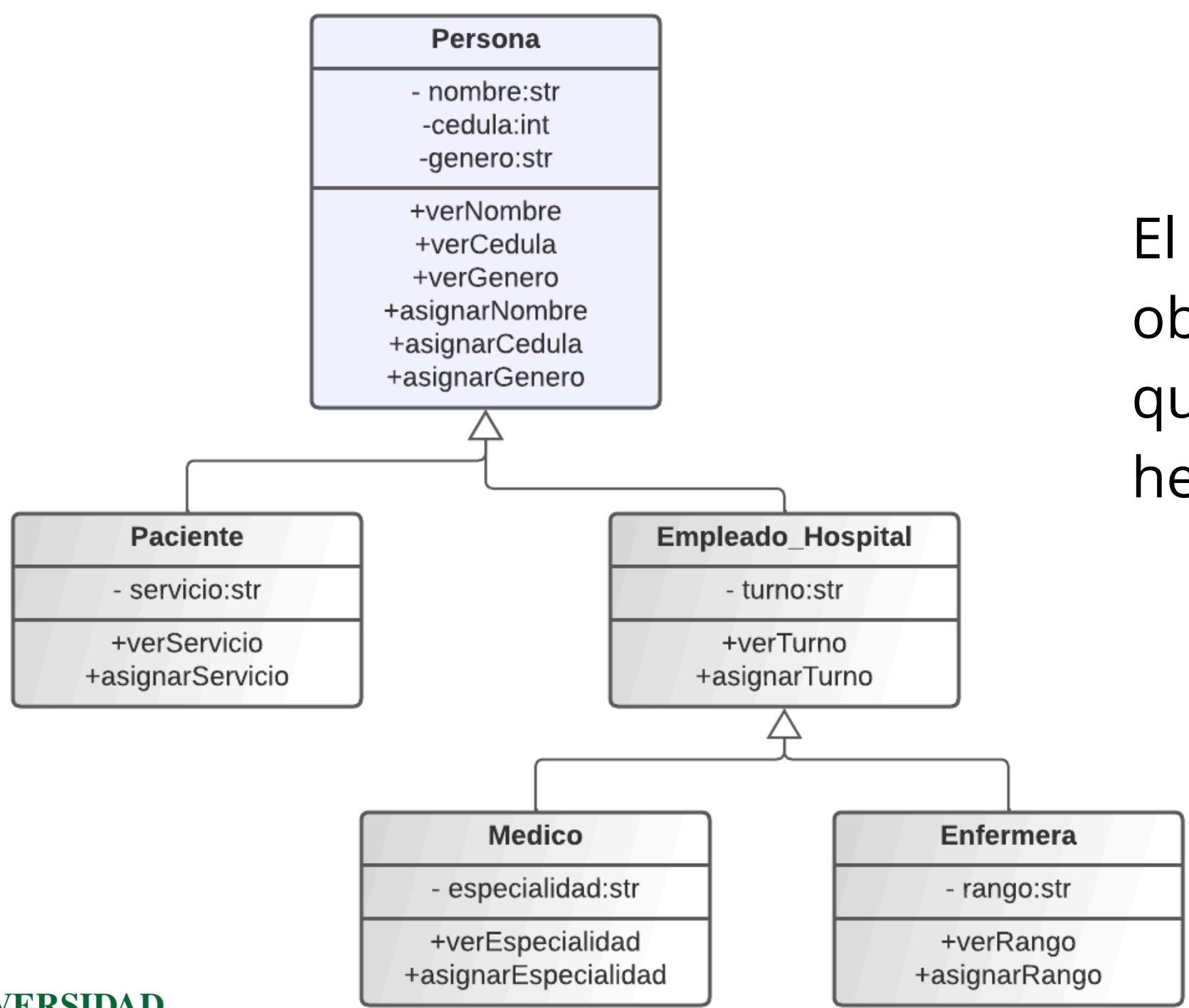
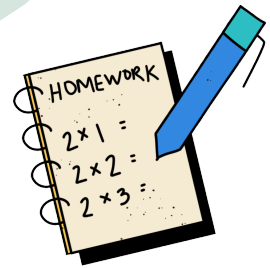


Diagrama UML

El siguiente diagrama muestra las clases obtenidas y la relación entre las mismas que en este caso es puramente de herencia.

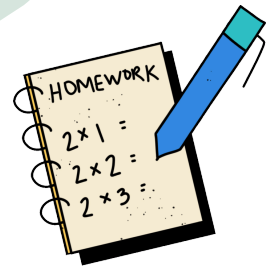




1er Ejercicio de Abstracción

Persona
- nombre:str -cedula:int -genero:str
+ operation1(params) # operation2(params) - operation3()

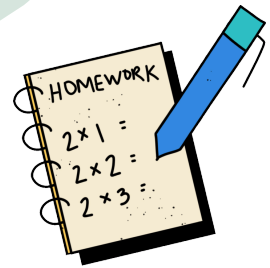
```
3 class Persona():
4     def __init__(self):
5         self.__nombre = ""
6         self.__cedula = 0
7         self.__genero = ""
```



1er Ejercicio de Abstracción

Código Python, añadamos métodos para acceder a los atributos.

Persona
- nombre:str -cedula:int -genero:str
+verNombre +verCedula +verGenero +asignarNombre +asignarCedula +asignarGenero

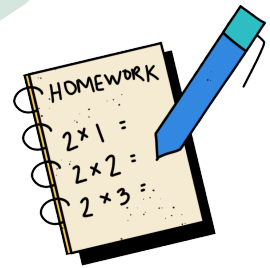


1er Ejercicio de Abstracción

Código Python, añadamos métodos para acceder a los atributos. DIGITAR

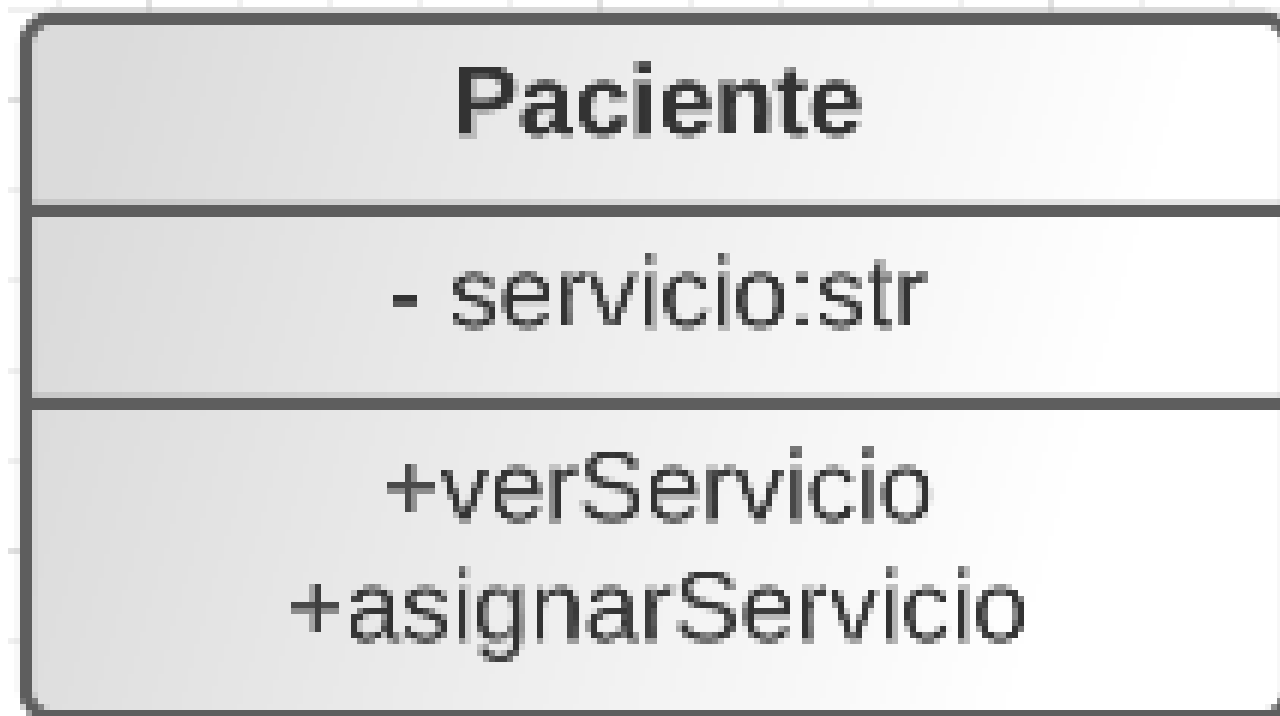
```
9      # Setters
10  ✓    def asignarNombre(self,nombre):
11      self.__nombre = nombre
12  ✓    def asignarCedula(self,cedula):
13      self.__cedula = cedula
14  ✓    def asignarGenero(self,genero):
15      self.__genero = genero
```

```
17      # getters
18  ✓    def verNombre(self):
19      return self.__nombre
20  ✓    def verCedula(self):
21      return self.__cedula
22  ✓    def verGenero(self):
23      return self.__genero
```



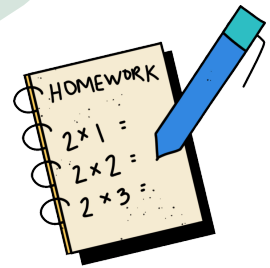
1er Ejercicio de Abstracción

Diagrama de clase



Código PYTHON

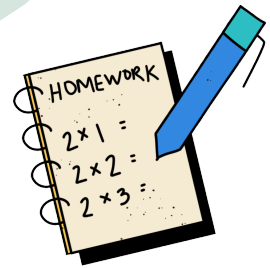
```
35 class Paciente(Persona):
36     def __init__(self):
37         self.__servicio = ""
38     def asignarServicio(self, servicio):
39         self.__servicio = servicio
40     def verServicio(self, servicio):
41         return self.__servicio
```



1er Ejercicio de Abstracción

Aprovechemos que hay dos clases para ver cómo opera la herencia. Creamos un objeto Paciente y le asignamos nombre, cédula y género usando los métodos que heredó de la clase Persona

```
p1 = Paciente()  
p2 = Paciente()  
p1.asignarNombre("Pepito")  
p1.asignarCedula(123413)  
p1.asignarGenero("Mujer")  
print(p1.verNombre())
```

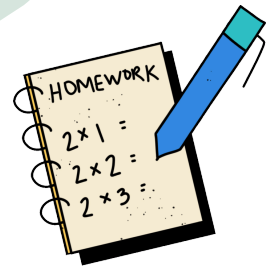
1er Ejercicio de Abstracción

Diagrama de clase

Código PYTHON

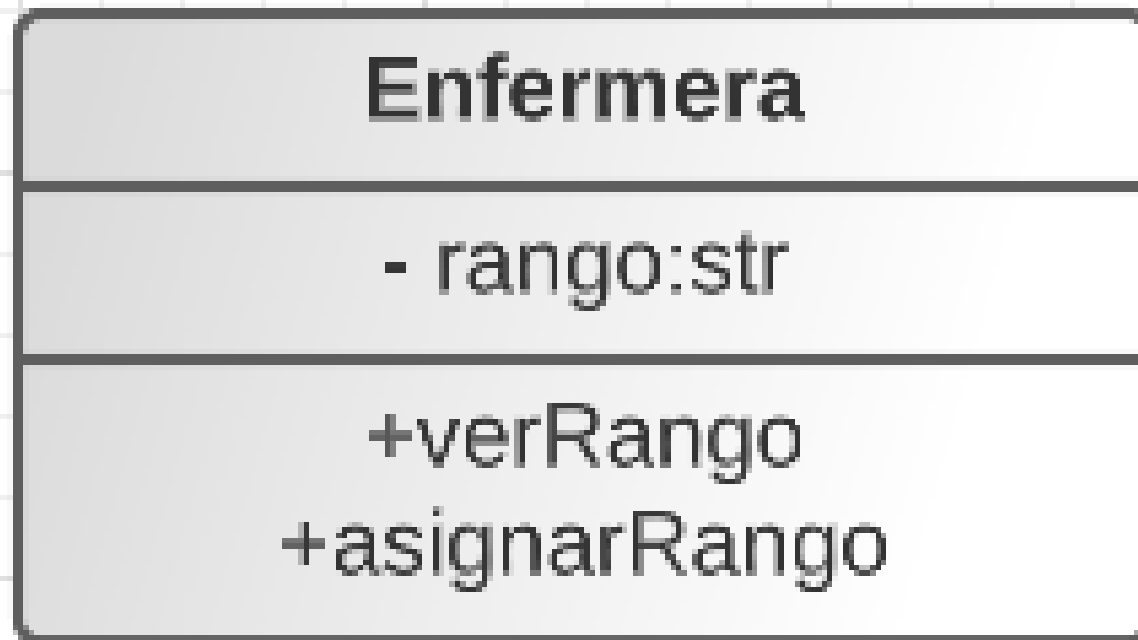


```
class Empleado_Hospital(Persona):  
    def __init__(self):  
        self.__turno = ''  
    def asignarTurno(self, turno):  
        self.__turno = turno  
    def verturno(self, turno):  
        return self.__turno
```



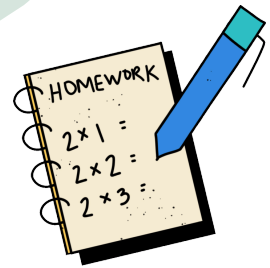
1er Ejercicio de Abstracción

Diagrama de clase



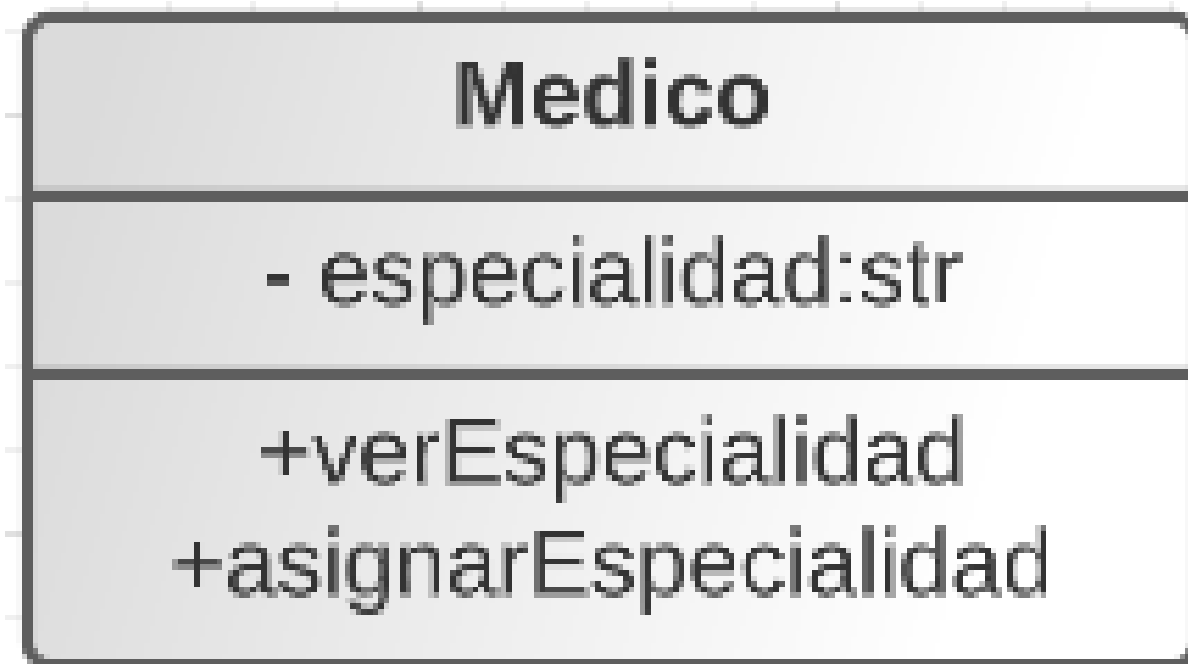
Código PYTHON

```
class Enfermera(Empleado_Hospital):  
    def __init__(self):  
        self.__rango = ''  
    def asignarRango(self, rango):  
        self.__rango = rango  
    def verRango(self, rango):  
        return self.__rango
```



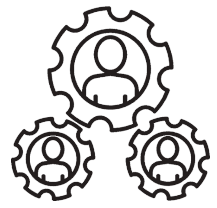
1er Ejercicio de Abstracción

Diagrama de clase



Código PYTHON

```
class Medico(Empleado_Hospital):  
    def __init__(self):  
        self.__especialidad = ''  
    def asignarEspecialidad(self, especialidad):  
        self.__especialidad = ''  
    def verEspecialidad(self, especialidad):  
        return self.__especialidad
```



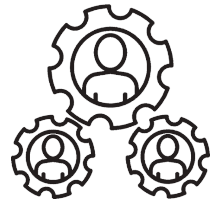
Para la clase

Ámbito de las variables:

- Donde existen todas las variables?
- En qué momento existen los atributos?
- Los parámetros de una función en qué momento existen?

```
1  import math
2  class Geometria2D(object):
3      def __init__(self):
4          self.__area=0
5          self.__perimetro = 0
6          #dijimos que la geometría tiene
7          #un conjunto o lista de puntos
8          self.__puntos = []
9
10     def verArea(self):
11         return self.__area
12
13     def asignarArea(self,a):
14         self.__area = a
15
```



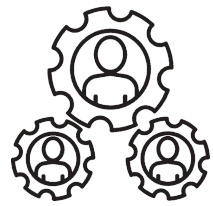


Para la clase

Ámbito de las variables:

- Donde existen todas las variables?
- En qué momento existen los atributos?
- Los parámetros de una función en qué momento existen?

```
16 class Circulo(Geometria2D):
17     def __init__(self):
18         Geometria2D.__init__(self)
19         self.__radio = 0.0;
20     def asignarRadio(self,r):
21         self.__radio = r;
22     def calcularArea(self):
23         #la calculo
24         a = math.pi*(self.__radio*self.__radio);
25         #actualizo el atributo
26         self.asignarArea(a);
```

Para la clase

- Como python combina programación estructurada con objetos la variable 'a' existe dentro del constructor pero esto es un error de lógica y debe evitarse definir variables por fuera de las clases y antes de las mismas
- También deben evitarse las variables globales
- Este programa generará error?
- Ver:<http://radek.io/2011/07/21/static-variables-and-methods-in-python/>
- <https://docs.python.org/es/3/tutorial/classes.html>

```
1  a = 10
2  class Prueba():
3
4      def __init__(self):
5          self.__b = a
6          temp = a
7          print(temp)
8
9      def verB(self):
10         return self.__b;
11
12     def verC(self):
13         return self.temp
```

```
1  o = Prueba()
2  print(o.verB())
3  print(o.verC())
```





Problema adicional

Para el material adelantado antes de la clase construyamos un menú que permita ingresar los datos ya sea de un paciente, una enfermera o un médico. Una vez ingresados imprima por pantalla la información respectiva

